Laboratorio PMDM UT5

La Ruta hacia la Montaña del Destino y La Marcha de los Ents a Isengard



2º DAM
Guillermo Martínez Martínez
GitHub: https://github.com/guillemtnz/LaboratorioUT5PMDM

ÍNDICE

1. Configuración del proyecto **2**
2. Etapa **2**
3. EtapaAdapter **2**
4. SplashActivity **2**
5. MainActivity **2**
5.1 Lista **2**
5.2 Menú de opciones **2**
5.3 Menú contextual **3**
5.4 Menú emergente **3**
6. DetalleActivity **3**
7. Ciclo de vida **3**
8. Capturas de pantalla **4**

1. Configuración del proyecto

Esta aplicación está hecha en Kotlin con **`Views`** y tiene tres pantallas: **`SplashActivity`**, **`MainActivity`** y **`DetalleActivity`**. Es la unión de los dos laboratorios: el listado de etapas con sus menús y la optimización del adaptador con **`ViewHolder`**.

2. Etapa

`Etapa` es una **`data class`** que representa cada fila de la lista. Tiene tres atributos: **`nombre`**, **`peligro`** (ambos **`String`**) e **`imagenResId`**, el identificador numérico del recurso gráfico.

```kotlin
package com.example.labut5

8 Usages
data class Etapa(
    val nombre: String,
    val peligro: String,
    val imagenResId: Int
)
```

3. EtapaAdapter

`EtapaAdapter` hereda de **`BaseAdapter`** e implementa **`getCount`**, **`getItem`**, **`getItemId`** y **`getView`**. Para optimizar la lista aplico el patrón **`ViewHolder`**: una clase interna que guarda las referencias a **`ivTerreno`**, **`tvNombre`** y **`tvPeligro`**, de forma que **`findViewById`** solo se llama una vez por fila.

Cuando **`convertView`** es **`null`** se infla **`item_etapa.xml`**, se crea el **`ViewHolder`** y se guarda en **`view.tag`**. Cuando la vista se reutiliza, se recupera el holder con **`view.tag as ViewHolder`** en lugar de volver a buscar las vistas.

```kotlin
3 Usages
private class ViewHolder(view: View) {
    1 Usage
    val ivTerreno: ImageView = view.findViewById(R.id.ivTerreno)
    1 Usage
    val tvNombre: TextView = view.findViewById(R.id.tvNombre)
    1 Usage
    val tvPeligro: TextView = view.findViewById(R.id.tvPeligro)
}
```

```kotlin
override fun getView(position: Int, convertView: View?, parent: ViewGroup?): View {
    val view: View
    val holder: ViewHolder

    if (convertView == null) {
        // Inflar y guardar holder
        view = LayoutInflater.from(context).inflate( resource = R.layout.item_etapa, root = parent, attachToRoot = false)
        holder = ViewHolder(view)
        view.tag = holder
    } else {
        // Reutilizar holder
        view = convertView
        holder = view.tag as ViewHolder
    }

    val etapa = listaEtapas[position]

    holder.ivTerreno.setImageResource(etapa.imagenResId)
    holder.tvNombre.text = etapa.nombre
    holder.tvPeligro.text = "Peligrosidad: {etapa.peligro}"

    return view
}
```

4. SplashActivity

La Splash muestra un **`ProgressBar`** y usa un **`Handler`** ligado al **`Looper`** principal para esperar 3 segundos con **`postDelayed`**. Después lanza un **`Intent`** hacia **`MainActivity`** y llama a **`finish()`**, para que al pulsar el botón Atrás no se vuelva a la pantalla de carga.

```kotlin
1 Usage
class SplashActivity : AppCompatActivity() {

    6 Usages
    private val tag = "SplashActivity"
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_splash)

        Handler(Looper.getMainLooper()).postDelayed( r = {
            startActivity(Intent( packageContext = this, cls = MainActivity::class.java))
            finish()
        }, delayMillis = 3000)
    }
```

5. MainActivity

5.1 Lista

En el **`onCreate`** creo la lista de etapas, se la paso a **`EtapaAdapter`** y se la asigno al **`ListView`** (**`lvEtapas`**).

```
etapas = listOf(
    Etapa( nombre = "La Comarca", peligro = "Baja", imagenResId = android.R.drawable.ic_menu_myplaces),
    Etapa( nombre = "Rivendell", peligro = "Baja", imagenResId = android.R.drawable.ic_menu_compass),
    Etapa( nombre = "Minas de Moria", peligro = "Alta", imagenResId = android.R.drawable.ic_menu_mapmode),
    Etapa( nombre = "Lothlórien", peligro = "Media", imagenResId = android.R.drawable.ic_menu_gallery),
    Etapa( nombre = "Emyn Muil", peligro = "Media", imagenResId = android.R.drawable.ic_menu_directions),
    Etapa( nombre = "Mordor", peligro = "Extrema", imagenResId = android.R.drawable.ic_dialog_alert)
)

lvEtapas.adapter = EtapaAdapter( context = this, listaEtapas = etapas)
```

5.2 Menú de opciones

`menu_principal.xml` tiene un ítem padre, **`Misión`**, con dos subítems: **`Abandonar Misión`** y **`Reiniciar Ruta`**. **`onCreateOptionsMenu`** infla el menú y **`onOptionsItemSelected`** gestiona los clics. **`Abandonar Misión`** cierra la app con **`finishAffinity()`** y **`Reiniciar Ruta`** solo muestra un **`Toast`**.

```
// Menú de opciones
override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    menuInflater.inflate( menuRes = R.menu.menu_principal, menu)
    return true
}

override fun onOptionsItemSelected(item: MenuItem): Boolean {
    return when (item.itemId) {
        R.id.opc_abandonar -> {
            finishAffinity()
            true
        }
        R.id.opc_reiniciar -> {
            Toast.makeText( context = this, resId = R.string.toast_reiniciar, duration = Toast.LENGTH_SHORT).show()
            true
        }
        else -> super.onOptionsItemSelected(item)
    }
}
```

5.3 Menú contextual

Para que la pulsación larga funcione hay que registrar el **`ListView`** con **`registerForContextMenu(lvEtapas)`**. **`onCreateContextMenu`** infla **`menu_contextual_etapa.xml`** y **`onContextItemSelected`** recupera la fila pulsada con **`AdapterContextMenuInfo.position`**. Al elegir **`Inspeccionar Terreno`** se lanza un **`Intent`** explícito hacia **`DetalleActivity`** con el nombre de la etapa y su peligro como extras.

```kotlin
// Menú contextual
override fun onCreateContextMenu(
    menu: ContextMenu?,
    v: View?,
    menuInfo: ContextMenu.ContextMenuInfo?
) {
    super.onCreateContextMenu(menu, v, menuInfo)
    menuInflater.inflate( menuRes = R.menu.menu_contextual_etapa, menu)
}

override fun onContextItemSelected(item: MenuItem): Boolean {
    val info = item.menuInfo as AdapterView.AdapterContextMenuInfo
    val etapa = etapas[info.position]

    return when (item.itemId) {
        R.id.ctx_inspeccionar -> {
            val intent = Intent( packageContext = this,  cls = DetalleActivity::class.java).apply {
                putExtra( name = "EXTRA_NOMBRE",  value = etapa.nombre)
                putExtra( name = "EXTRA_PELIGRO",  value = etapa.peligro)
            }
            startActivity(intent)
            true
        }
        else -> super.onContextItemSelected(item)
    }
}
```

5.4 Menú emergente

El botón **`btnAccionesMarcha`** crea un **`PopupMenu`** anclado a sí mismo en su **`setOnClickListener`**. Se infla **`menu_popup_marcha.xml`**, se escuchan los clics con **`setOnMenuItemClickListener`** y al final hay que llamar a **`popup.show()`**. Cada opción solo muestra un **`Toast`**.

```kotlin
// Menú emergente
findViewById<Button>(R.id.btnAccionesMarcha).setOnClickListener { viewAncla ->
    val popup = PopupMenu( context = this,  anchor = viewAncla)
    popup.menuInflater.inflate( menuRes = R.menu.menu_popup_marcha, popup.menu)
    popup.setOnMenuItemClickListener { menuItem ->
        when (menuItem.itemId) {
            R.id.pop_acelerar -> {
                Toast.makeText( context = this,  resId = R.string.toast_acelerar,  duration = Toast.LENGTH_SHORT).show()
                true
            }
            R.id.pop_acampar -> {
                Toast.makeText( context = this,  resId = R.string.toast_acampar,  duration = Toast.LENGTH_SHORT).show()
                true
            }
            else -> false
        }
    }
    popup.show()
}
```

6. DetalleActivity

`DetalleActivity` lee los extras del **`Intent`** con **`getStringExtra`**, usando **`Desconocido`** como valor por defecto, y los muestra en dos **`TextView`**.

```
6 Usages
private val tag = "DetalleActivity"
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_detalle)

    val nombre = intent.getStringExtra( name = "EXTRA_NOMBRE") ?: "Desconocido"
    val peligro = intent.getStringExtra( name = "EXTRA_PELIGRO") ?: "Desconocido"

    findViewById<TextView>(R.id.tvDetalleNombre).text = nombre
    findViewById<TextView>(R.id.tvDetallePeligro).text =
        "Peligrosidad: {peligro}"
}
```

7. Ciclo de vida

Las tres activities sobrescriben **`onStart`**, **`onResume`**, **`onPause`**, **`onStop`**, **`onRestart`** y **`onDestroy`**, además de **`onCreate`**. Cada método llama a **`super`** y escribe una línea en Logcat con **`Log.d`**, usando el nombre de la activity como etiqueta, así se puede ver el orden en el que se llaman al navegar entre pantallas.

```
override fun onStart() {
    super.onStart()
    Log.d(tag, msg = "onStart")
}

override fun onResume() {
    super.onResume()
    Log.d(tag, msg = "onResume")
}

override fun onPause() {
    super.onPause()
    Log.d(tag, msg = "onPause")
}

override fun onStop() {
    super.onStop()
    Log.d(tag, msg = "onStop")
}

override fun onRestart() {
    super.onRestart()
    Log.d(tag, msg = "onRestart")
}

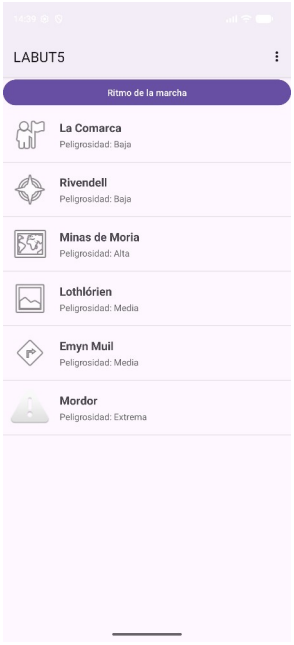
override fun onDestroy() {
    super.onDestroy()
    Log.d(tag, msg = "onDestroy")
}
```

8. Capturas de pantalla

Estas son las capturas de la aplicación ejecutándose en el emulador: la Splash, el listado, el detalle, los tres tipos de menú y un **`Toast`**.



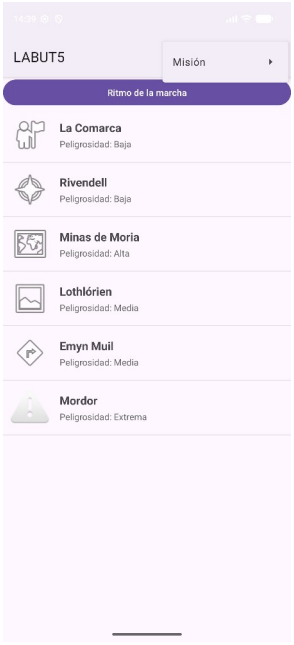
Splash



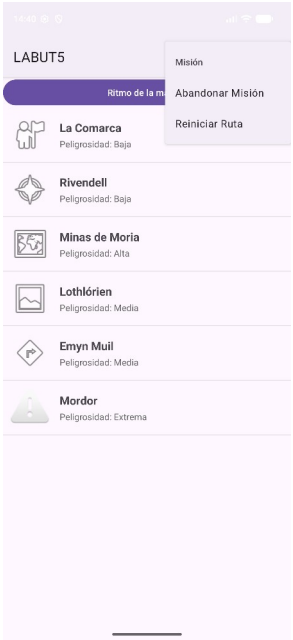
Listado de etapas



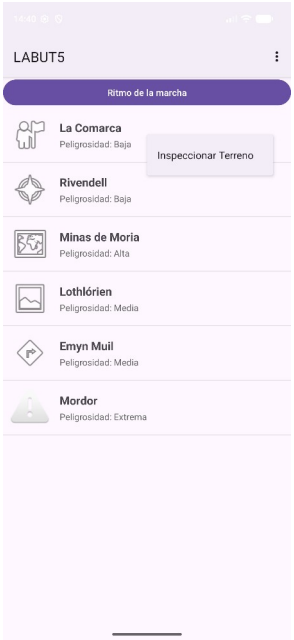
Detalle



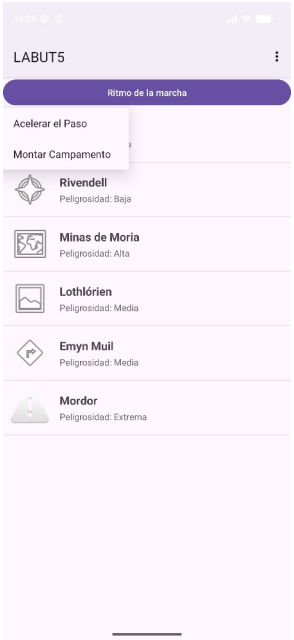
Menú de opciones



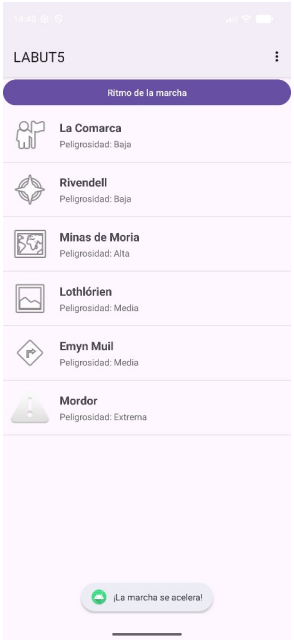
Submenú de Misión



Menú contextual



Menú emergente



Toast